

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	URHYNIX	팀 인 원 (팀장 포함)	2명
참가부문	일반	과제유형	자유과제

□ 결과보고서

프로젝트 개요	
프로젝트명	UnityCTL - AI 에이전트를 위한 Unity 에디터 제어 도구
프로젝트 등록 URL	https://github.com/Jason-hub-star/unityctl
시연영상	https://youtu.be/OJOgWHOZV3Q
프로젝트 소개	AI 에이전트가 Unity 에디터를 직접 조작하도록 연결해 주는 오픈소스 CLI 이자 MCP 서버입니다. 썬과 게임오브젝트를 다루고 플레이모드를 돌리는 일부터 스크린샷으로 결과를 확인하는 것까지, 170개가 넘는 명령을 명령줄과 CI에서 그대로 쓸 수 있습니다.
프로젝트 세부 내용	
개발배경 및 목적	- 요즘 AI 코딩 도구는 웹이나 앱은 곧잘 짜주지만, 정작 게임을 만드는 Unity 에디터 안으로는 손을 뻗지 못합니다. - 2026년 Unity가 'Unity AI'를 내놨지만, 어디까지나 에디터 안에서 사람이 대화로 쓰는 보조 기능이고 특정 구독에 묶여 있습니다. - 외부의 AI 에이전트가 명령줄이나 CI에서 에디터를 직접, 그것도 일관된 결과로 제어할 방법은 여전히 비어 있었습니다. - UnityCTL은 바로 그 자리를 채웁니다. 어떤 LLM이든 자연어로 Unity 를 부르고 결과까지 스스로 확인하게 하는, 특정 회사에 묶이지 않는 오픈소스를 목표로 만들었습니다.
개발환경	- 런타임: .NET 10 (SDK 10.0.201), Shared 모듈은 .NET Standard 2.1 - 언어: C# (CLI·Core·MCP)와 Unity Editor 플러그인(UPM) - CLI 프레임워크: ConsoleAppFramework 5.3.3, 출력 렌더링은 Spectre.Console - AI 연동: ModelContextProtocol 1.1.0 기반 MCP 서버 - 직렬화: System.Text.Json(CLI/Core), Newtonsoft.Json(Plugin), 스크린샷 처리는 SixLabors.ImageSharp - 테스트와 CI: xUnit 887개, GitHub Actions 3-OS 매트릭스 (Windows·macOS·Linux) - 지원 Unity 버전: 2021.3 이상
시스템 구성 및 아키텍처	- 계층은 Shared(프로토콜·모델)를 Core(전송·탐색·재시도)가 받고, 그 위에 얇은 CLI를 얹는 구조입니다. MCP 서버는 별도로 둡니다.

	- Unity 플러그인은 에디터 안에서 도는 IPC 서버 역할을 합니다. - 전송은 IPC(Named Pipe)를 먼저 찢어보고, 안 되면 batchmode 실행으로 자동으로 넘어갑니다. - IPC 프레이밍은 [4바이트 LE 길이][UTF-8 JSON] 형태로, 최대 10MB 까지 받습니다. - 도메인 리로드로 연결이 끊기는 구간은 ipc-state.json 하트비트와 reload 인지 재시도로 메웁니다. - screenshot과 workflow verify로 에이전트가 자기 작업 결과를 직접 눈으로 확인하는 페루프를 갖췄습니다.
프로젝트 주요기능	프로젝트 상세 내용 - 170개가 넘는 CLI 명령: 씬 계층과 게임오브젝트·컴포넌트 조작, 플레이모드, 스크린샷, 스크립트 생성·편집·패치, 프로파일러, 머터리얼·애니메이션·UI(UGUI/UI Toolkit)·Cinemachine까지 다룹니다. - MCP 하이브리드: 통합 도구 몇 개로 수많은 명령을 토큰 효율적으로 노출합니다(33개를 12개로 묶음). - 멀티 에디터 라우팅: editor current/select로 여러 인스턴스를 골라 보낼 수 있습니다. - 운영 기능: 비동기 명령, 세션 레이어, Flight Recorder, Ghost Mode(--dry-run), Watch Mode를 갖췄습니다. - 시각 검증 워크플로우: workflow verify와 playSmoke로 게임이 실제로 도는 장면을 캡처해 판정합니다. 구동 및 시연 - 설치: NuGet으로 CLI를 받고, Unity 프로젝트에 UPM 플러그인을 추가합니다. - 실행: dotnet run --project src/Unityctl.Cli -- <command>로 직접 쓰거나, MCP 서버로 Claude·Cursor 등과 연결합니다. - 검증: dotnet test 887개가 통과하고 3-OS CI가 초록불을 유지합니다.
기대효과 및 활용분야	향후 확장성 및 기대효과 - 반복되는 씬 구성과 컴포넌트 설정, 빌드 점검을 자연어 한 줄로 대신하니 개발 속도가 붙습니다. - Unity API를 깊이 몰라도 말로 프로토타입을 만들 수 있어, 1인 개발자나 교육 현장에서 특히 쓸모가 큼니다. - AI와 게임엔진을 잇는 표준 MCP 인터페이스를 887개 테스트와 CI까지 통째로 공개합니다. - Claude든 Cursor든 가리지 않고 붙고, 기존 CI/CD 파이프라인에도 그대로 엹니다. - 스크린샷으로 결과를 스스로 확인하는 구조라, 코드만 뺀고 끝나는 게 아니라 실제로 돌아가는 게임까지 끌고 갑니다.
기타	프로젝트의 혁신성 및 차별성 - Unity AI는 에디터 창 안에서 대화로만 쓰고 구독이 필요하지만, UnityCTL은 외부 AI가 헤드리스 환경이나 CI에서 에디터를 일관되게 제어합니다. - MIT 라이선스 오픈소스라 특정 LLM이나 구독에 발이 묶이지

않습니다.

- CLI와 MCP를 분리했고, IPC를 먼저 시도하다 안 되면 batchmode로 알아서 떨어집니다.
- 스크린샷 기반 시각 검증과 여러 에디터 인스턴스 라우팅을 함께 갖췄습니다.
- 887개 테스트와 3개 OS CI로 완성도를 숫자로 보여줍니다.

한계점 및 향후 발전 로드맵

- 아직 UI Toolkit처럼 손이 덜 닿은 도메인이 남아 있어 명령 커버리지를 더 넓혀야 합니다.
- Unity 버전과 렌더 파이프라인별 호환성은 추가 검증이 필요합니다.
- 앞으로는 명령 범위 확대, 시각 검증 고도화, 설치·문서 온보딩 개선을 차례로 풀어갈 계획입니다.

소감 및 후기

- 'AI한테 게임 만드는 손을 쥐여주자'는 한 줄 아이디어로 시작했는데, 막상 Unity 에디터를 외부에서 안정적으로 붙드는 일이 생각보다 험난했습니다. 도메인 리로드 한 번에 연결이 통째로 끊기던 문제를 하트비트 상태 파일로 잡아낸 순간이 가장 기억에 남습니다.
 - 둘이서 한 명은 Core 전송 계층을, 한 명은 Unity 플러그인을 맡아 IPC 양쪽을 맞춰가는 과정이 곧 협업 그 자체였습니다. 테스트를 887개까지 쌓아두니 리팩터링이 두렵지 않았고, 그 안정감이 끝까지 밀어붙이는 힘이 됐습니다.
 - 오픈소스로 공개하며 받은 피드백 하나하나가 방향을 잡아줬습니다.
- 이번 대회를 계기로 AI와 게임 개발이 만나는 길을 더 많은 분과 함께 넓혀가고 싶습니다.

붙임1

SBOM(소프트웨어 자재명세서)

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	ConsoleAppFramework	5.3.3	MIT	https://github.com/Cysharp/ConsoleAppFramework	CLI 명령 등록·라우팅 프레임워크
2	ModelContextProtocol	1.1.0	MIT	https://github.com/modelcontextprotocol/csharp-sdk	MCP 서버 구현(AI 에이전트 연동)
3	Spectre.Console	0.49	MIT	https://github.com/spectreconsole/spectre.console	CLI 출력·테이블 렌더링
4	SixLabors.ImageSharp	3.1.12	Six Labors Split License (오픈소스는 Apache-2.0)	https://github.com/SixLabors/ImageSharp	스크린샷 이미지 인코딩·처리
5	System.Text.Json	8.0.5	MIT	https://github.com/dotnet/runtime	CLI/Core JSON 직렬화(Source Generator)
6	Newtonsoft.Json (Unity)	3.2.1	MIT	https://github.com/JamesNK/Newtonsoft.Json	Unity 플러그인 JSON 직렬화

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

- ☐ 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
 - ☐ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정한 경우)
 - ☐ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
- ※ 개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI(GPT, Claude 등)를 단순 활용한 경우는 체크하지 않음(4번 항목에 기재)

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (AI 모델 미탑재)	기반 모델 라이선스	해당 없음
--------------	-------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	해당 없음 (외부·자체 AI 모델 미사용)
데이터 정제/ 가공 방법 요약	해당 없음 (외부·자체 AI 모델 미사용)
새로 생성된 가중치 공개 저장소 URL	해당 없음 (외부·자체 AI 모델 미사용)
가중치 파일 정보 및 배포방식	해당 없음 (외부·자체 AI 모델 미사용)

4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)

직접 작성한 코드의 오픈소스 라이선스	MIT License (OSI 인증)	학습/추론 소스코드 공개 저장소 URL	해당 없음 (AI 모델 학습·추론 코드 없음)
상용 AI 보조도구 활용 여부 및 범위	코드 작성과 디버깅 보조용으로 Anthropic Claude, OpenAI GPT 등 상용 AI를 활용했습니다. 다만 프로젝트에 AI 모델을 탑재하거나 학습하지 않으므로 유형 1·2·3에는 해당하지 않습니다.		